

RICK AND MORTY DATABASE

Feld	Angabe
Projektname	Rick and Morty Database
Projektart	React-Webapplikation mit externer REST-API und lokaler json-server-Datenbank
Teammitglieder	Silvio Apa, Nicola Schmied
Klasse	UIFZ 2524-021
Lehrperson	Graziano Laveder
Abgabedatum	06.06.2026, - 15:00 Uhr
GitHub-Repository	https://github.com/silvio-apa/RM-DB



Table of Contents

1. Projektidee
2. Anforderungskatalog
3. Klassendiagramm / Datenmodell
4. Storyboard
5. Screen-Mockups
6. REST-Schnittstellen
7. Testplan
8. Testergebnisse
9. Unit-Tests
10. Installationsanleitung
11. Hilfestellungen / Quellen
12. Projektverlauf / Git-Pushes
13. Fazit
14. Arbeitsaufteilung



1. Projektidee

Unsere Projektidee ist eine Webapplikation im Stil eines Pokédex, jedoch für Charaktere aus der Serie Rick and Morty. Die Applikation zeigt eine seitenweise Übersicht der Charaktere an. Benutzerinnen und Benutzer können nach Charakteren suchen, zwischen mehreren Ergebnisseiten wechseln und Detailinformationen zu einzelnen Charakteren öffnen.

Die Charakterdaten werden über die öffentliche Rick-and-Morty-API geladen. Zusätzlich verwendet das Projekt eine eigene lokale REST-API mit json-server. Darin werden Favoriten gespeichert. Zu jedem Favoriten kann außerdem eine persönliche Notiz erfasst und später wieder geändert oder gelöscht werden.

Das Ziel des Projekts ist es, eine übersichtliche und einfach bedienbare React-App zu entwickeln, welche externe API-Daten mit eigenen gespeicherten Daten kombiniert. Die App soll zeigen, dass wir mit React, React Router, REST-Schnittstellen, json-server, CRUD-Operationen, Suche, Pagination, Fehlerhandling und Unit Tests arbeiten können.



2. Anforderungskatalog

2.1 User Story 1: Charakterübersicht anzeigen

Als Benutzer möchte ich eine Übersicht aller Rick-and-Morty-Charaktere sehen, damit ich schnell durch die verfügbaren Charaktere browsen kann.

Akzeptanzkriterien

- Die App zeigt mehrere Charaktere als Karten an.
- Jede Karte enthält Bild, Name, Status und Species.
- Die Charakterdaten werden aus der Rick-and-Morty-API geladen.
- Während des Ladens erscheint ein Loading-Hinweis.
- Wenn keine Daten geladen werden können, erscheint eine Fehlermeldung mit Try-Again-Möglichkeit.

2.2 User Story 2: Charaktere suchen und Seiten wechseln

Als Benutzer möchte ich Charaktere nach Namen suchen und durch Ergebnisseiten wechseln können, damit ich gezielt Charaktere finde.

Akzeptanzkriterien

- Es gibt ein Suchfeld auf der Characters-Seite.
- Die Suche wird über den Namen an die API gesendet.
- Die Suche setzt die aktuelle Seite wieder auf Seite 1 zurück.
- Es gibt Previous- und Next-Buttons für die Pagination.
- Wenn keine Treffer gefunden werden, erscheint der Hinweis No characters found.

2.3 User Story 3: Charakterdetails anzeigen

Als Benutzer möchte ich einen Charakter anklicken können, damit ich weitere Informationen zu diesem Charakter sehe.

Akzeptanzkriterien

- Jede Charakterkarte besitzt einen View-Details-Link.
- Die Detailseite wird über `/characters/:id` geöffnet.
- Die Detailseite zeigt Name, Bild, Status, Species, Gender, Origin und Location.
- Es gibt einen Back-to-Characters-Link.
- Wenn die Details nicht geladen werden können, erscheint eine verständliche Fehlermeldung.



2.4 User Story 4: Favoriten speichern

Als Benutzer möchte ich Charaktere als Favoriten speichern können, damit ich meine Lieblingscharaktere später schnell wiederfinde.

Akzeptanzkriterien

- Auf der Detailseite gibt es einen Add-to-Favorites-Button.
- Ein Favorit wird per POST in der lokalen json-server-Datenbank gespeichert.
- Die App prüft, ob ein Charakter bereits gespeichert ist.
- Wenn ein Charakter bereits gespeichert ist, wird der Button deaktiviert.
- Nach dem Speichern erscheint eine Bestätigungsmeldung.

2.5 User Story 5: Favoriten verwalten

Als Benutzer möchte ich meine Favoriten anzeigen, mit Notizen ergänzen und löschen können, damit ich meine persönliche Sammlung verwalten kann.

Akzeptanzkriterien

- Die Favorites-Seite lädt alle gespeicherten Favoriten aus der lokalen API.
- Zu jedem Favoriten wird Bild, Name, Status und Species angezeigt.
- Zu jedem Favoriten kann eine persönliche Notiz gespeichert werden.
- Ein Favorit kann gelöscht werden.

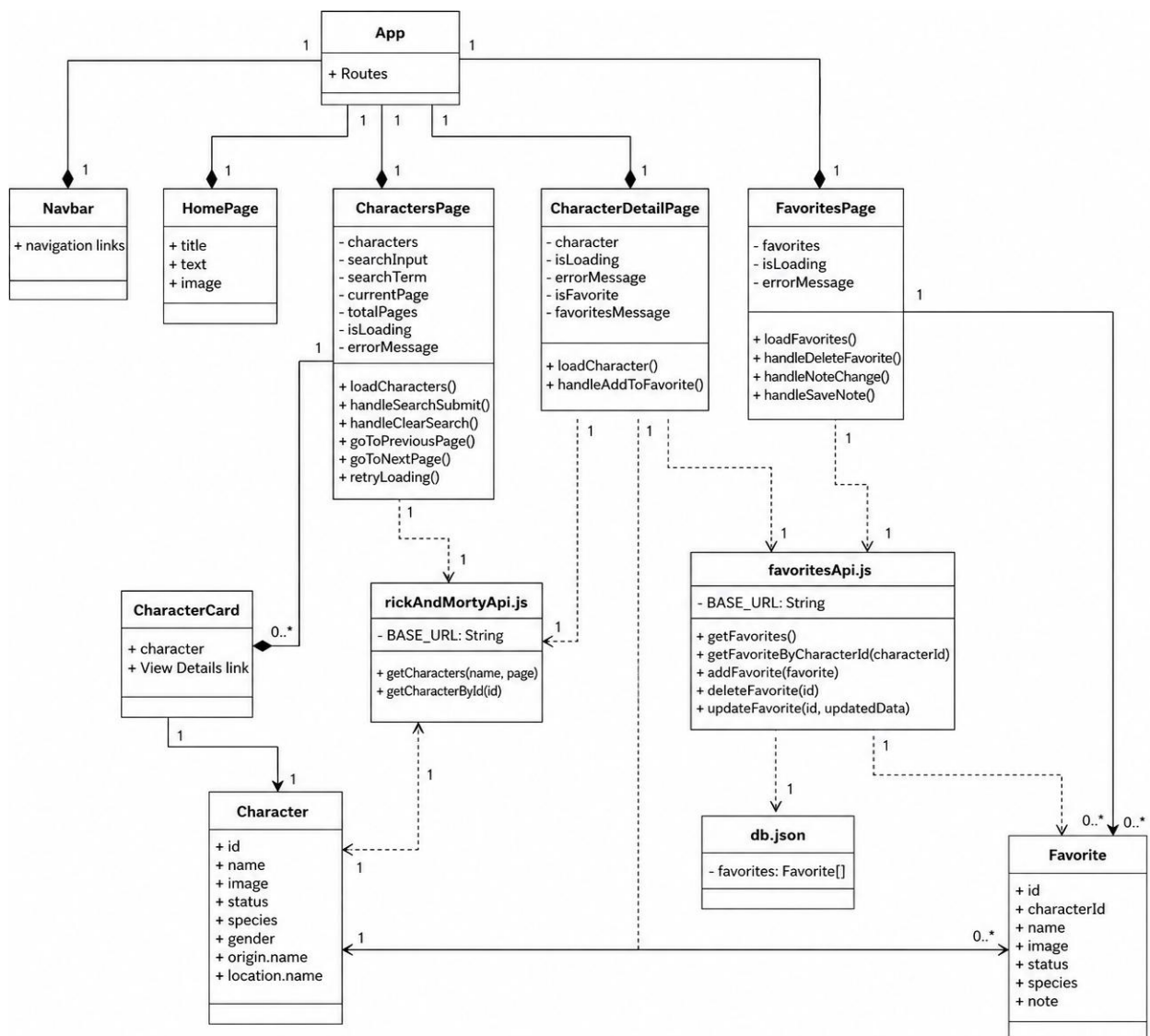
Bei API-Fehlern erscheint eine verständliche Fehlermeldung.



3. Klassendiagramm / Datenmodell

Das Projekt arbeitet mit Daten aus der externen Rick-and-Morty-API und mit eigenen gespeicherten Favoriten aus der lokalen json-server-Datenbank. Da es sich um ein React-Projekt handelt, bestehen die wichtigsten Elemente nicht aus klassischen Java-Klassen, sondern aus React-Komponenten, Service-Dateien und Datenobjekten.

Für das Datenmodell sind vor allem die Objekte Character und Favorite wichtig. Zusätzlich werden die Service-Dateien rickAndMortyApi.js und favoritesApi.js beschrieben, weil sie für die REST-Anfragen zuständig sind.



Klasse / Datenobjekt: Character

Diese Klasse beschreibt einen Charakter aus der Rick-and-Morty-API.

Attribut	Datentyp	Beschreibung
id	Number	Eindeutige ID des Charakters aus der API
name	String	Name des Charakters
status	String	Status des Charakters, z. B. Alive, Dead oder unknown
species	String	Spezies des Charakters
gender	String	Geschlecht des Charakters
image	String	URL zum Bild des Charakters
origin.name	String	Herkunft des Charakters
location.name	String	Aktueller Aufenthaltsort des Charakters

Klasse / Datenobjekt: Favorite

Diese Klasse beschreibt einen gespeicherten Favoriten in unserer eigenen lokalen Datenbank db.json.

Attribut	Datentyp	Beschreibung
id	String / Number	Von json-server automatisch erzeugte ID des Favoriten-Eintrags
characterId	Number	ID des Charakters aus der Rick-and-Morty-API
name	String	Name des gespeicherten Charakters
image	String	Bild-URL des gespeicherten Charakters
status	String	Status des Charakters
species	String	Spezies des Charakters
note	String	Optionale persönliche Notiz zum Charakter



Service-Datei: rickAndMortyApi.js

Diese Service-Datei ist für die Anfragen an die externe Rick-and-Morty-API zuständig.

Attribut / Methode	Datentyp	Beschreibung
BASE_URL	String	Basis-URL der Rick-and-Morty-API
getCharacters(name, page)	Function	Lädt Charaktere aus der API. Optional können ein Suchbegriff und eine Seitenzahl übergeben werden.
getCharacterById(id)	Function	Lädt einen einzelnen Charakter anhand seiner ID aus der API.

Service-Datei: favoritesApi.js

Diese Service-Datei ist für die Anfragen an die lokale json-server-API zuständig.

Attribut / Methode	Datentyp	Beschreibung
BASE_URL	String	Basis-URL der lokalen Favoriten-API
getFavorites()	Function	Lädt alle gespeicherten Favoriten aus db.json.
getFavoriteByCharacterId(characterId)	Function	Prüft, ob ein bestimmter Charakter bereits als Favorit gespeichert ist.
addFavorite(favorite)	Function	Speichert einen neuen Favoriten per POST in der lokalen Datenbank.
deleteFavorite(id)	Function	Löscht einen Favoriten per DELETE aus der lokalen Datenbank.
updateFavorite(id, updatedData)	Function	Aktualisiert einen Favoriten per PATCH, zum Beispiel die persönliche Notiz.



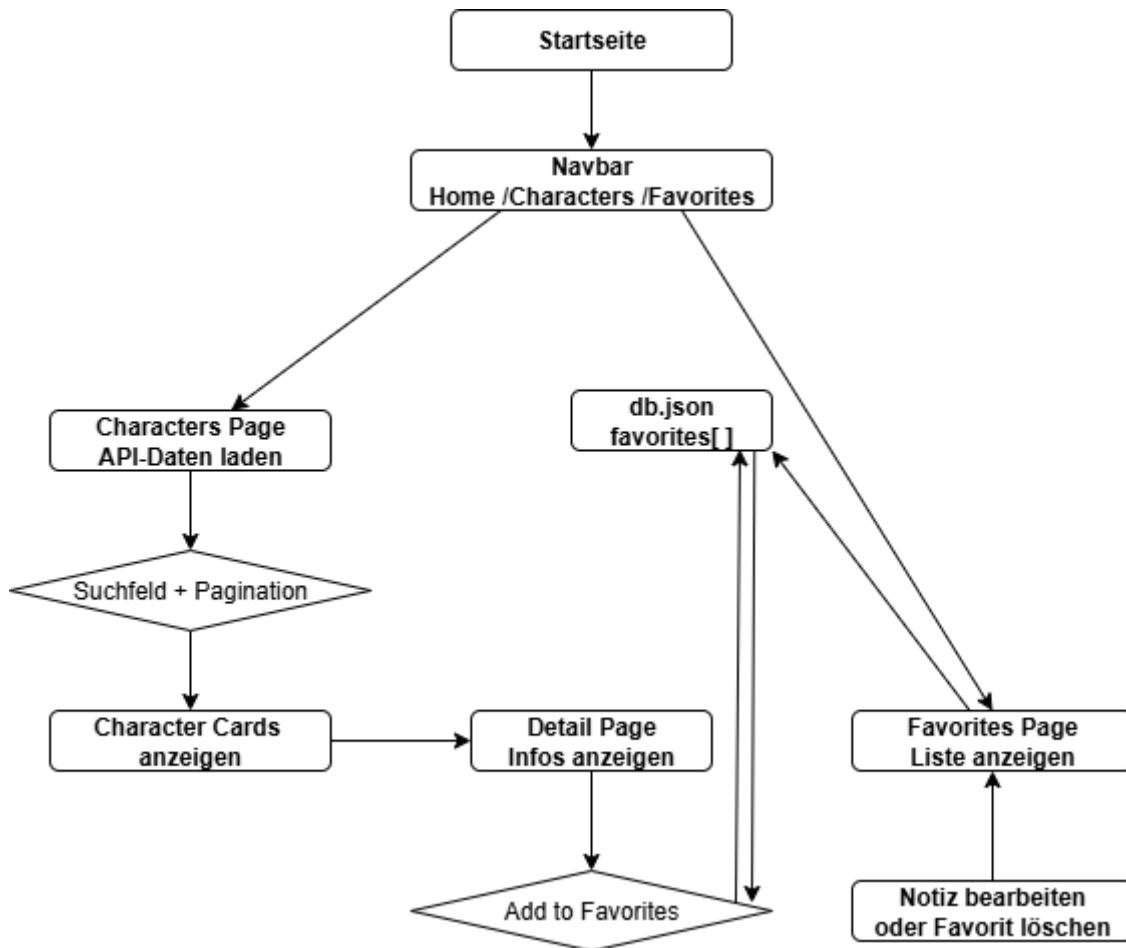
Beziehungen im Datenmodell

Verbindung	Kardinalität	Bedeutung
Character → Favorite	1 → 0..1	Ein Charakter kann kein Favorit sein oder einmal als Favorit gespeichert werden.
Favorite → Character	1 → 1	Jeder Favorit gehört über characterId zu genau einem Charakter.
favoritesApi.js → Favorite	1 → 0..*	Die Service-Datei verwaltet keine, einen oder mehrere Favoriten.
db.json → Favorite	1 → 0..*	Die Datei db.json enthält eine Liste von Favoriten.
rickAndMortyApi.js → Character	1 → 0..*	Die Service-Datei lädt keine, einen oder mehrere Charaktere aus der externen API.
CharacterCard → Character	1 → 1	Eine CharacterCard zeigt genau einen Charakter an.
CharacterDetailPage → Character	1 → 1	Die Detailseite zeigt genau einen ausgewählten Charakter an.
FavoritesPage → Favorite	1 → 0..*	Die FavoritesPage zeigt keine, einen oder mehrere gespeicherte Favoriten an.



4. Storyboard

Das Storyboard zeigt den Navigationsfluss durch die Applikation.



Schematischer Ablauf

4.1 Beschreibung des Ablaufs

1. Der Benutzer öffnet die Webapplikation.
2. Die Startseite wird angezeigt.
3. Über die Navigation wechselt der Benutzer zur Characters-Seite.
4. Die App lädt Character-Daten aus der Rick-and-Morty-API.
5. Die Charaktere werden als Karten angezeigt.
6. Der Benutzer kann einen Suchbegriff eingeben und die Ergebnisse werden gefiltert.
7. Über Previous und Next kann der Benutzer zwischen Ergebnisseiten wechseln.
8. Der Benutzer öffnet über View Details die Detailseite eines Characters.
9. Die Detailseite zeigt zusätzliche Informationen an.
10. Der Benutzer speichert den Character als Favorite.
11. Der Favorite wird per POST in db.json gespeichert.
12. Auf der Favorites-Seite werden gespeicherte Favorites angezeigt.
13. Der Benutzer kann eine persönliche Notiz speichern oder einen Favorite löschen.



4.2 Entscheidungspunkte

- Wird ein Suchbegriff eingegeben? Ja: API-Suche nach Name; Nein: allgemeine Character-Liste.
- Gibt es Suchergebnisse? Ja: Cards anzeigen; Nein: No characters found anzeigen.
- Ist der Character bereits Favorit? Ja: Button zeigt Already in Favorites; Nein: Add to Favorites ist möglich.
- Gibt es gespeicherte Favorites? Ja: Liste anzeigen; Nein: Hinweis No favorites saved yet anzeigen.
- Tritt ein API-Fehler auf? Ja: verständliche Fehlermeldung anzeigen; Nein: Daten normal darstellen.



5. Screen-Mockups

Die folgenden Wireframes zeigen die geplanten bzw. im Projekt vorhandenen Screens. Die sichtbaren Texte der App sind auf Englisch, weil die API-Daten ebenfalls auf Englisch sind. (Die Mockups können vom Ergebnis abweichen da sie zu Beginn des Projektes entstanden sind und der Hauptfokus im Projekt nicht auf der Darstellung lag.)

Screen-Mockups / Wireframes

Rick and Morty Database – schwarz/grau im Stil der Dokumentation

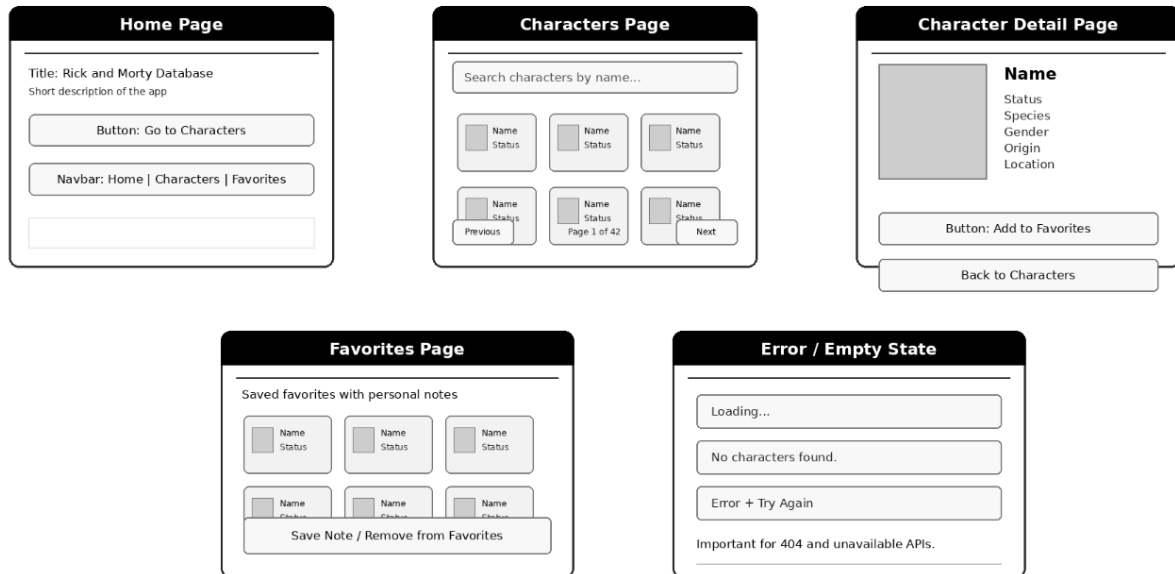


Abbildung 3: Wireframes der wichtigsten Screens.

Screen	Inhalt	Zweck
Home Page	Startscreen mit Titel, kurzer Beschreibung und Navbar.	Einstieg und Erklärung der App.
Characters Page	Suchfeld, Character Cards, Loading/Error/Empty-State, Pagination.	Übersicht und Suche nach Characters.
Character Detail Page	Bild, Name, Status, Species, Gender, Origin, Location, Add-to-Favorites-Button.	Detailinformationen anzeigen und Favorite speichern.
Favorites Page	Liste gespeicherter Favorites, Notizfeld, Save Note, Remove from Favorites.	Persönliche Sammlung verwalten.
Error / Empty State	No characters found, Loading-Meldungen und Error-Meldungen.	Fehlerhandling verständlich darstellen.



6. REST-Schnittstellen

Die Applikation verwendet zwei REST-Schnittstellen: die externe Rick-and-Morty-API für Character-Daten und eine lokale json-server-API für Favorites.

6.1 Externe API: Rick and Morty API

Eigenschaft	Wert
Basis-URL	https://rickandmortyapi.com/api
Service-Datei	src/services/rickAndMortyApi.js
Zweck	Characters laden, nach Namen suchen, Details anhand einer ID laden

Datentyp: Character

```
{
  "id": 1,
  "name": "Rick Sanchez",
  "status": "Alive",
  "species": "Human",
  "gender": "Male",
  "origin": { "name": "Earth (C-137)" },
  "location": { "name": "Citadel of Ricks" },
  "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg"
}
```

Feld	Typ	Beschreibung	Pflicht
id	Number	Eindeutige Character-ID	ja
name	String	Name des Characters	ja
status	String	Alive, Dead oder unknown	ja
species	String	Spezies	ja
gender	String	Geschlecht	ja
origin.name	String	Herkunft	ja
location.name	String	Aktueller Aufenthaltsort	ja
image	String	Bild-URL	ja



Methode	Endpoint	Beschreibung	Response / Fehler
GET	/character/?page={page}	Lädt Characters einer bestimmten Ergebnisseite.	200 OK, Objekt mit info und results; Fehler bei Serverproblem.
GET	/character/?name={name}&page={page}	Sucht Characters nach Namen und unterstützt Pagination.	200 OK mit results; bei 404 wird im Frontend eine leere Ergebnisliste angezeigt.
GET	/character/{id}	Lädt Details zu einem Character anhand der ID.	200 OK mit Character-Objekt; 404 bei ungültiger ID.



6.2 Lokale REST-API: Favorites mit json-server

Eigenschaft	Wert
Basis-URL	http://localhost:3001
Datenbank-Datei	db.json
Ressource	favorites
Service-Datei	src/services/favoritesApi.js
Startbefehl	npm run server

```
{
  "favorites": []
}
```

Datentyp: Favorite

```
{
  "id": "1",
  "characterId": 1,
  "name": "Rick Sanchez",
  "image": "https://rickandmortyapi.com/api/character/avatar/1.jpeg",
  "status": "Alive",
  "species": "Human",
  "note": "Main character and scientist"
}
```

Feld	Typ	Beschreibung	Pflicht
id	String / Number	Von json-server automatisch erzeugte ID	automatisch
characterId	Number	ID des Characters aus der externen API	ja
name	String	Name des Characters	ja
image	String	Bild-URL des Characters	ja
status	String	Status des Characters	ja
species	String	Spezies des Characters	ja
note	String	Persönliche Notiz	nein



Methode	Endpoint	Beschreibung	Response / Fehler
GET	/favorites	Lädt alle gespeicherten Favorites.	200 OK, Array von Favorite-Objekten; 500 bei Serverproblem.
GET	/favorites?characterId={id}	Prüft, ob ein bestimmter Character bereits als Favorite gespeichert ist.	200 OK, Array; leer, falls nicht vorhanden.
POST	/favorites	Speichert einen neuen Favorite.	201 Created, gespeichertes Objekt inkl. ID; 400 bei ungültigem Body.
PATCH	/favorites/{id}	Aktualisiert Teilinformationen eines Favorites, aktuell die persönliche Notiz.	200 OK, aktualisiertes Objekt; 404 bei ungültiger ID.
DELETE	/favorites/{id}	Löscht einen gespeicherten Favorite.	200 OK oder 204 No Content; 404 bei ungültiger ID.



7. Testplan

Der Testplan dokumentiert manuelle Tests, die nach der Implementierung durchgeführt werden. Die tatsächlichen Ergebnisse werden im nächsten Kapitel eingetragen. Die Tests orientieren sich an den Funktionen, die im aktuellen Projektcode vorhanden sind.

ID	Titel	Vorbedingung	Schritte	Erwartetes Ergebnis
T1	Applikation starten	Repository wurde geklont und npm install wurde ausgeführt.	1. npm run dev starten. 2. Browser-URL öffnen.	Die Home Page wird ohne Fehler angezeigt.
T2	Lokalen Server starten	Abhängigkeiten sind installiert.	1. npm run server starten. 2. http://localhost:3001/favorites öffnen.	json-server läuft und zeigt die favorites-Ressource an.
T3	Characters anzeigen	Frontend läuft und externe API ist erreichbar.	1. Characters öffnen. 2. Warten, bis Daten geladen sind.	Character Cards mit Bild, Name, Status und Species werden angezeigt.
T4	Pagination verwenden	Characters-Seite zeigt Ergebnisse an.	1. Next klicken. 2. Previous klicken.	Die Seite wechselt korrekt und zeigt Page X of Y an.
T5	Character suchen	Characters-Seite ist geöffnet.	1. Suchbegriff Rick eingeben.	Nur passende Characters werden angezeigt, ohne komplettes Neuladen der App.
T6	Ungültige Suche	Characters-Seite ist geöffnet.	1. Ungültigen Namen eingeben, z. B. xyzabc123.	No characters found wird angezeigt, die App stürzt nicht ab.
T7	Detailseite öffnen	Character Cards werden angezeigt.	1. View Details bei einem Character klicken.	Die Detailseite mit korrekten Daten wird geöffnet.
T8	Favorite speichern	Detailseite ist geöffnet und json-server läuft.	1. Add to Favorites klicken. 2. Favorites öffnen.	Character erscheint in der Favorites-Liste.
T9	Doppelten Favorite verhindern	Character wurde bereits gespeichert.	1. Detailseite desselben Characters erneut öffnen.	Button zeigt Already in Favorites und ist deaktiviert.
T10	Notiz speichern	Mindestens ein Favorite existiert.	1. Favorites öffnen. 2. Notiz eingeben. 3. Save Note klicken.	Notiz wird per PATCH gespeichert und bleibt erhalten.
T11	Favorite löschen	Mindestens ein Favorite existiert.	1. Remove from Favorites klicken.	Favorite verschwindet aus der Liste und aus db.json.
T12	Fehlerhandling lokaler Server	Frontend läuft, json-server ist gestoppt.	1. Favorites öffnen.	Fehlermeldung Failed to load favorites erscheint, kein weißer Bildschirm.



8. Testergebnisse

ID	Testfall	Status	Bemerkung
T1	Applikation starten	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T2	Lokalen Server starten	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T3	Characters anzeigen	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T4	Pagination verwenden	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T5	Character suchen	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T6	Ungültige Suche	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T7	Detailseite öffnen	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T8	Favorite speichern	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T9	Doppelten Favorite verhindern	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T10	Notiz speichern	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T11	Favorite löschen	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.
T12	Fehlerhandling lokaler Server	OK	Nach finalem lokalen Test auf OK / Nicht OK setzen.



9. Unit-Tests

ID	Unit-Test	Beschreibung
UT1	CharacterCard rendert Daten	Eine CharacterCard erhält ein Character-Objekt und zeigt Name, Bild, Status und Species.
UT2	Detail-Link enthält ID	Der Link in CharacterCard führt zu <code>/characters/{id}</code> .
UT3	getCharacters baut Such-URL korrekt	Bei Suchbegriff Rick wird <code>/character/?name=Rick&page=1</code> aufgerufen.
UT4	404-Suche liefert leere Resultate	Wenn die externe API 404 zurückgibt, liefert getCharacters ein leeres results-Array.
UT5	addFavorite sendet POST	addFavorite sendet einen POST-Request an <code>/favorites</code> mit JSON-Body.
UT6	updateFavorite sendet PATCH	updateFavorite sendet eine PATCH-Anfrage an <code>/favorites/{id}</code> .
UT7	FavoritesPage löscht Favorite	Nach deleteFavorite wird der gelöschte Favorite nicht mehr angezeigt.



10. Installationsanleitung

10.1 Voraussetzungen

- Node.js installiert
- npm installiert
- Git installiert
- Visual Studio Code oder eine andere IDE
- Internetverbindung für die Rick-and-Morty-API

10.2 Projekt herunterladen

```
git clone https://github.com/silvio-apa/RM-DB.git  
cd RM-DB
```

10.3 Abhängigkeiten installieren

```
npm install
```

10.4 Lokale Favorites-API starten

Terminal 1 im Projektordner öffnen und folgenden Befehl ausführen:

```
npm run server
```

Danach läuft json-server unter <http://localhost:3001>. Die Favorites können im Browser unter <http://localhost:3001/favorites> geprüft werden.

10.5 React-Frontend starten

Terminal 2 im Projektordner öffnen und folgenden Befehl ausführen:

```
npm run dev
```

Danach wird im Terminal eine lokale Vite-URL angezeigt, normalerweise <http://localhost:5173>. Diese URL im Browser öffnen.

10.6 Unit Tests ausführen

Terminal 3 im Projektordner öffnen und folgenden Befehl ausführen:

```
npm run test:run
```



10.7 Funktionstest nach der Installation

- Home Page öffnet sich.
- Navbar wechselt zwischen Home, Characters und Favorites.
- Characters-Seite lädt Daten aus der Rick-and-Morty-API.
- Suche und Pagination funktionieren.
- Detailseite öffnet den richtigen Character.
- Add to Favorites speichert in db.json.
- Favorites-Seite zeigt gespeicherte Favorites.
- Notizen können gespeichert werden.
- Favorites können gelöscht werden.



11. Hilfestellungen / Quellen

Quelle / Person	Art der Hilfe
Projektpartner	Zusammenarbeit bei Planung, Umsetzung und Kontrolle des Projektstands.
Lehrperson	Fachliche Unterstützung, Anforderungen und Bewertungskriterien.
Rick-and-Morty-API-Dokumentation: https://rickandmortyapi.com/documentation	Informationen zu Character-Endpunkten, Suche, Pagination und Detaildaten.
React-Dokumentation: https://react.dev	Informationen zu Komponenten, State und Hooks.
React Router: https://reactrouter.com	Informationen zu Routes, NavLink, Link und useParams.
Vite-Dokumentation: https://vite.dev	Informationen zum Projektsetup und Entwicklungsserver.
KI-Tools	- Unterstützung bei Struktur, Formulierungen, Abgleich und Dokumentationsentwurf. - Unterstützung bei Fehler Behebung im Code. Vor allem bei Pagination. - Generell wurde die KI für Fehler Behebung und bei Stecken bleiben im Prozess zugezogen. - CSS wurde von KI erstellt, weil die Darstellung nicht Projektrelevant war.



12. Projektverlauf / Git-Pushes

In diesem Kapitel wird der Projektverlauf anhand der wichtigsten Git-Pushes dokumentiert. Die Pushes zeigen, wie das Projekt Schritt für Schritt aufgebaut und erweitert wurde.

12.1 Push 1: React Project Setup

Durchgeführt

- React-Projekt mit Vite erstellt
- Grunddateien und Projektsetup generiert
- npm install ausgeführt
- Erstes rohes Setup auf GitHub gepusht

12.2 Push 2: Add Routing and Base Pages

Durchgeführt

- Projektstruktur vorbereitet
- Ordner components, pages, services und styles erstellt
- React Router installiert
- main.jsx mit BrowserRouter ergänzt
- App.jsx mit Routes aufgebaut
- Navbar erstellt
- Grundseiten erstellt:
 - Home
 - Characters
 - Character Detail
 - Favorites
- Navigation zwischen den Seiten getestet



12.3 Push 3: Display Characters from API with Cards

Durchgeführt

- Rick-and-Morty-API angebunden
- API-Service rickAndMortyApi.js vorbereitet und genutzt
- Komponente CharacterCard.jsx erstellt
- CharactersPage lädt echte Charakterdaten aus der API
- Charaktere werden als Karten angezeigt
- Karten zeigen Bild, Name, Status und Species
- View Details-Link führt zur Detailseite mit passender Character-ID

12.4 Push 4: Implement Character Detail Page

Durchgeführt

- CharacterDetailPage funktionsfähig umgesetzt
- Character-ID wird mit useParams aus der URL gelesen
- getCharacterById(id) lädt den passenden Charakter aus der Rick-and-Morty-API
- Detailseite zeigt:
 - Bild
 - Name
 - Status
 - Species
 - Gender
 - Origin
 - Location
- Loading-State eingebaut
- Error-Message eingebaut
- Back-Link zur CharactersPage hinzugefügt



12.5 Push 5: Add Favorite Characters

Durchgeführt

- json-server als lokale REST-Datenbank eingerichtet
- db.json mit favorites-Collection erstellt
- package.json um Server-Script ergänzt
- favoritesApi.js erstellt
- Folgende Favoriten-Funktionen umgesetzt:
 - getFavorites
 - getFavoriteByCharacterId
 - addFavorite
 - deleteFavorite
- CharacterDetailPage um Add to Favorites-Button erweitert
- FavoritesPage lädt gespeicherte Favoriten aus json-server
- Favoriten werden als Karten angezeigt
- Favoriten können wieder gelöscht werden

12.6 Push 6: Add Favorite Notes Editing

Durchgeführt

- favoritesApi.js um updateFavorite mit PATCH erweitert
- Favoriten können aktualisiert werden
- FavoritesPage zeigt ein Notizfeld pro Favorit
- Notizen werden in db.json gespeichert
- Nach dem Neuladen bleiben gespeicherte Notizen erhalten
- Styling für Notizfeld und Button ergänzt
- Favoriten können weiterhin gelöscht werden



12.7 Push 7: Add Character Search and Pagination

Durchgeführt

- getCharacters erweitert, sodass Suchbegriff und Seitenzahl an die Rick-and-Morty-API übergeben werden können
- CharactersPage mit Suchfeld für Namen ergänzt
- Bei neuer Suche wird automatisch wieder auf Seite 1 gewechselt
- Pagination mit Previous- und Next-Button eingebaut
- Aktuelle Seite und Gesamtseitenzahl werden angezeigt
- Buttons werden während dem Laden deaktiviert bzw. abgesichert
- Bei keinen Treffern erscheint eine passende Meldung
- Bei API-Fehler erscheint eine Fehlermeldung

12.8 Push 8: Improve Favorite Note Saving

Durchgeführt

- Notizen werden nicht mehr bei jedem Tastendruck gespeichert
- Textarea ändert zuerst nur den lokalen React-State
- Save Note-Button speichert die Notiz gezielt mit PATCH in db.json
- Leere Notizen können gespeichert werden, damit bestehende Notizen entfernt werden können
- Fehlermeldungen auf der FavoritesPage wurden verbessert
- Save- und Delete-Buttons wurden gestylt



12.9 Push 9: Improve Error Handling and Code Formatting

Durchgeführt

- Fehlermeldungen wurden benutzerfreundlicher formuliert
- Technische error.message Ausgaben wurden auf den Seiten durch verständlichere Meldungen ersetzt
- CharactersPage, CharacterDetailPage und FavoritesPage zeigen einheitlichere Fehlermeldungen
- Code in betroffenen Dateien formatiert
- Lesbarkeit und Struktur des Codes verbessert

12.10 Push 10: Improve Character Search Behavior

Durchgeführt

- Live-Suche wurde durch ein Suchformular ersetzt
- Search-Button hinzugefügt
- Clear-Button hinzugefügt
- API-Anfragen werden nicht mehr bei jedem Tastendruck ausgelöst
- searchInput speichert den aktuellen Text im Suchfeld
- searchTerm wird erst beim Absenden an die API übergeben
- Clear-Button setzt Suche und Seite zurück
- Pagination bleibt mit der Suche kombiniert
- Detailseiten Cleanup mitgepusht:
 - Komponentename von CharacterDetailsPage zu CharacterDetailPage vereinheitlicht
 - CSS-Klasse von character-details zu character-detail angepasst



12.11 Push 11: Add Unit Tests

Durchgeführt

- Vitest installiert und eingerichtet
- React Testing Library eingerichtet
- Test-Setup mit jsdom und jest-dom erstellt
- Test-Script in package.json ergänzt
- UT1 bis UT7 umgesetzt
- Tests prüfen:
 - UT1: CharacterCard rendert Charakterdaten
 - UT2: Detail-Link enthält die richtige Character-ID
 - UT3: getCharacters baut die Such-URL korrekt
 - UT4: 404-Suche liefert leere Resultate
 - UT5: addFavorite sendet POST-Request
 - UT6: updateFavorite sendet PATCH-Request
 - UT7: FavoritesPage löscht einen Favoriten
- Alle 7 Tests laufen erfolgreich mit `npm run test:run`

12.12 Push 12: Add Home Page Image

Durchgeführt

- Bild im Ordner `public/images` abgelegt
- HomePage um ein großes Bild unter Titel und Text erweitert
- Styling für HomePage ergänzt
- Styling für Textbereich ergänzt
- Styling für Bild ergänzt
- Titelaufstand und Lesbarkeit verbessert
- Startseite wirkt dadurch vollständiger und ansprechender



12.13 Push 13: Add README Instructions and Project Documentation

Durchgeführt

- Standard-Vite-README ersetzt
- Eigenes README für das Projekt erstellt
- Projekt kurz beschrieben
- Features aufgelistet
- Installationsanleitung ergänzt
- Startanleitung für React-App ergänzt:
 - `npm run dev`
- Startanleitung für lokale Datenbank ergänzt:
 - `npm run server`
- Hinweis ergänzt, dass React-App und json-server gleichzeitig laufen müssen
- Testanleitung ergänzt:
 - `npm run test:run`
- Projektdokumentation im GitHub-Root abgelegt
- Dokumentation für die Abgabe vorbereitet



13. Fazit

Der Rick and Morty Character Explorer ist eine React-Webapplikation, die externe API-Daten mit einer eigenen lokalen REST-API kombiniert. Die Anwendung zeigt Charaktere aus der Serie Rick and Morty in einer übersichtlichen Kartenansicht an. Benutzerinnen und Benutzer können nach Charakteren suchen, durch Ergebnisseiten navigieren, Detailinformationen öffnen und interessante Charaktere als Favoriten speichern.

Durch die lokale json-server-Datenbank können Favoriten gespeichert, bearbeitet und gelöscht werden. Damit erfüllt die App die wichtigsten CRUD-Anforderungen: Daten werden gelesen, neue Favoriten werden erstellt, persönliche Notizen werden aktualisiert und Favoriten können wieder gelöscht werden.

Während des Projekts konnten wichtige Themen aus Modul 294 praktisch angewendet werden. Dazu gehören React-Komponenten, Routing mit React Router, API-Services, REST-Schnittstellen, json-server, State-Management, Suche, Pagination, Fehlerhandling und Unit-Tests. Besonders wichtig war die Kombination aus externer Rick-and-Morty-API und eigener lokaler Favoriten-Datenbank.

Zusätzlich wurde der Code Schritt für Schritt verbessert. Dazu gehören benutzerfreundlichere Fehlermeldungen, eine stabilere Suche mit Search- und Clear-Button, eine bessere Speicherung der Favoriten-Notizen sowie lauffähige Unit-Tests mit Vitest und React Testing Library. Die Tests prüfen zentrale Funktionen wie Character Cards, API-Services, Favoriten-Requests und das Löschen von Favoriten.

Zusammenfassend wurde das Projekt erfolgreich umgesetzt. Die geplanten Hauptfunktionen sind vorhanden, die App ist lokal ausführbar, die wichtigsten Funktionen wurden getestet und die Dokumentation wurde mit dem aktuellen Projektstand abgeglichen.



14. Arbeitsaufteilung

Die Arbeit am Projekt wurde grundsätzlich gemeinsam durchgeführt. Wir haben die einzelnen Projektteile nicht strikt untereinander aufgeteilt, sondern das Projekt Schritt für Schritt zusammen geplant, umgesetzt und kontrolliert.

Zu Beginn haben wir gemeinsam die Projektidee definiert und entschieden, welche Funktionen unsere Rick and Morty Datenbank enthalten soll. Danach haben wir zusammen die Grundstruktur des React-Projekts erstellt, das Routing vorbereitet und die wichtigsten Seiten aufgebaut. Auch bei der API-Anbindung, der Darstellung der Character Cards, der Detailseite, der Favorites-Funktion, der lokalen json-server-Datenbank und dem Fehlerhandling haben wir jeweils gemeinsam gearbeitet.

Während der Umsetzung haben wir die nächsten Schritte regelmäßig miteinander besprochen. Wenn eine Funktion nicht direkt funktioniert hat, haben wir den Code zusammen überprüft, Fehler gesucht und Lösungen getestet. Dadurch waren beide Teammitglieder laufend über den gesamten Projektstand informiert und konnten bei allen wichtigen Bereichen mitarbeiten.

Auch die Dokumentation wurde gemeinsam abgestimmt. Inhalte wie Projektidee, Anforderungen, Datenmodell, REST-Schnittstellen, Testplan, Installationsanleitung und Projektverlauf wurden passend zum aktuellen Stand des Codes erstellt und kontrolliert.

Zusammenfassend wurde das Projekt als Teamarbeit umgesetzt. Beide Teammitglieder waren an Planung, Programmierung, Tests, Fehlerbehebung und Dokumentation beteiligt. Es gab keine komplett getrennten Einzelaufgaben, sondern eine gemeinsame schrittweise Entwicklung des gesamten Projekts.

